

Register Machine (machine)

Durante as obras de renovação das Casamatas de Bock – uma rede de fortificações subterrâneas na cidade do Luxemburgo – os trabalhadores desenterraram uma antiga arca do tesouro, selada por um mecanismo bastante peculiar.

A inscrição na tampa chama-lhe uma *máquina de registos*, e explica que tem 64 registos numerados de 0 a 63, cada um dos quais pode guardar um número inteiro não negativo. Inicialmente, todos os registos contêm 0.

Um conjunto de manípulos e botões permite-te introduzir um *programa* composto por uma sequência de instruções. A primeira instrução que introduzes tem a etiqueta 0, a instrução seguinte 1, e assim por diante. Existem três tipos de instruções:

- **INCREMENT** x, i — O valor guardado no registo x é aumentado em 1, e depois a máquina processa a instrução com a etiqueta i .
- **DECREMENT** x, i, j — Se o valor guardado no registo x for 0, a máquina processa a instrução com a etiqueta i . Caso contrário, o registo x é diminuído em 1, e depois a máquina processa a instrução com a etiqueta j .
- **HALT** — A máquina para de processar instruções.

A máquina começa a processar a partir da instrução com a etiqueta 0. Assim que parar, abrirá a arca se o valor guardado no registo 0 for exatamente N .

Se não parar após processar 50 000 000 instruções, ou se tentar processar uma instrução que não existe, a máquina sofrerá uma avaria catastrófica e selará a arca para sempre.

A tua tarefa é introduzir um programa que abra a arca. Como a máquina é delicada, queres que o comprimento do programa seja o mais curto possível, e certamente não superior a 100 instruções.

Implementação

Deves submeter um único ficheiro `.cpp` com o código-fonte.



Entre os anexos para este problema, encontrarás um modelo `machine.cpp` e um avaliador de exemplo `grader.cpp` que podes usar para testar as tuas soluções localmente.

Terás de implementar a seguinte função:

C++	<code>void program_machine(int N)</code>
-----	--

- O inteiro N representa o número a guardar no registo 0 quando a máquina parar.

O teu programa pode chamar as seguintes funções no máximo 100 vezes, as quais já estão definidas no avaliador (*grader*):

C++	<code>void add_increment(int x, int i)</code>
C++	<code>void add_decrement(int x, int i, int j)</code>
C++	<code>void add_halt()</code>

Avaliador Padrão

Entre os anexos para este problema, encontrarás uma versão simplificada do avaliador (*grader*) usado durante a avaliação, que podes utilizar para testar as tuas soluções localmente. O avaliador de exemplo lê os dados de `stdin`, chama a função `program_machine` e escreve de volta no `stdout` usando o seguinte formato.

O input é formado por uma linha, contendo o número inteiro N .

O output é formado por uma linha:

- Se o teu programa abrir a arca corretamente, o output é `OK L`, onde L é o comprimento do teu programa.
- Caso contrário, o output começa com `WA`, seguido de um breve motivo (ex.: o programa excedeu 100 instruções, o programa não parou a tempo, etc.).

Restrições

- $1 \leq N \leq 1000000$.

Pontuação

O teu programa será testado contra vários casos de teste agrupados em subtarefas. Para obteres a pontuação de uma subtarefa, o teu programa tem de abrir a arca corretamente em todos os casos de teste dessa subtarefa.

Para a subtarefa 3, a pontuação é determinada pelo comprimento do teu programa. Seja L o comprimento máximo do teu programa em todos os casos de teste desta subtarefa. A pontuação para esta subtarefa é $\min(77, 100 - L)$. Em particular, para obteres a totalidade dos 77 pontos nesta subtarefa, o teu programa tem de usar no máximo 23 instruções em todos os casos de teste desta subtarefa.

- **Subtarefa 0 [0 pontos]:** Casos de teste de exemplo.
- **Subtarefa 1 [7 pontos]:** $N \leq 99$.
- **Subtarefa 2 [16 pontos]:** N é uma potência de 2 (ou seja, $N = 2^k$ para algum inteiro $k \geq 0$).
- **Subtarefa 3 [77 pontos]:** Sem restrições adicionais. (Nota que a pontuação para esta subtarefa depende do comprimento do teu programa.)

Exemplos

Avaliador	Solução
<code>program_machine(3)</code>	<pre>add_increment(1, 1) add_increment(0, 2) add_decrement(1, 4, 1) add_halt() add_increment(0, 3)</pre>

Avaliador	Solução
<code>program_machine(5)</code>	<pre>add_increment(0, 1) add_increment(0, 2) add_increment(0, 3) add_increment(0, 4) add_increment(0, 5) add_halt()</pre>

Explicação

No primeiro exemplo, $N = 3$. O exemplo cria um programa com 5 instruções:

0.	INCREMENT 1, 1
1.	INCREMENT 0, 2
2.	DECREMENT 1, 4, 1
3.	HALT
4.	INCREMENT 0, 3

A tabela seguinte mostra a etiqueta de cada instrução que a máquina processa, e o conteúdo dos registos 0 e 1 de seguida:

Instrução	Registo 0	Registo 1
—	0	0
0	0	1
1	1	1
2	1	0
1	2	0
2	2	0
4	3	0
3	3	0

No segundo exemplo, $N = 5$ e o programa de exemplo apresentado utiliza 6 instruções.